Six-model B1 stack with calibrated threshold

Phase 4 Court Judgment Prediction | source-audited documentation

Problem statement and data

Phase 4 Court Judgment Prediction predicts binary outcome 0/1 from legal case text. The preserved official inputs are Train/train.json, Train/dev.json and Test/test.json. Test labels are private. Original run records report 994 official dev labels and 1,517 test IDs; macro-F1 is the central metric.

Architecture and mechanics

This is a prediction-level ensemble, not a newly trained Phase 1/B1 backbone. It joins saved calibration, dev and test probabilities from: v7_transformer, v6_transformer, v5_gru, v4_reference, v7_gru, v6_gru. An L2-regularized logistic regression learns the mixing weights from 512 training-calibration examples; each original model retains its own checkpoint. Decision threshold 0.514611. Data flow: B1 model probabilities -> 6 inputs -> 512 train calibration -> L2 logistic weights -> Threshold 0.515 -> Binary ZIP.

Scientific rationale

Prediction-level B1 ensemble, not an additional neural backbone. This experiment isolates the effect of its recorded adaptation or document-head mechanism while retaining the original task and export contract. Comparisons across different backbones, training budgets or data coverage remain descriptive rather than controlled causal estimates.

Stack mechanics and coefficient provenance

Input model names in order: v7_transformer, v6_transformer, v5_gru, v4_reference, v7_gru, v6_gru. Each input contributes a probability column for aligned case IDs. An L2-penalized logistic regression fits 512 train-calibration examples; the record stores coefficients [[0.6205551287478728, 0.22619116014049523, 0.180136918795945, 0.3552037848072189, 0.1385905289128631, 0.1385905289128631]] and intercept [-0.03948328269681701]. For each case, the fitted linear log-odds combine base probabilities, the logistic sigmoid maps the result to a score, and a fixed or calibration-selected threshold converts it to label 0/1. The score threshold is 0.514611; its selection rule and exact hashes for all input arrays are in the copied verification record.

Exact prediction path

For a case, stack the base-model probabilities into vector p. Compute logit = intercept + Σ(coefficient_i × p_i), apply sigmoid, and output 1 if the resulting probability reaches the recorded threshold. Every base probability comes from an independently trained B1 model; no backbone weight is updated when fitting the stack.

Validation and selection

Five-fold cross-validation over the 512 train-calibration cases yielded macro-F1 0.716744. Official-dev macro-F1 0.701977 on 994 cases is a separate report-only observation. Comparing several stack variants on the same dev data introduces post-selection optimism. The 1,517 test predictions were made without test labels.

Adaptation mechanism

L2-regularized logistic regression combines 6 aligned model probabilities: v7_transformer, v6_transformer, v5_gru, v4_reference, v7_gru, v6_gru. The stored coefficients are 0.6206, 0.2262, 0.1801, 0.3552, 0.1386, 0.1386 and intercept -0.0395.

Training design and checkpoint selection

Six B1 inputs; L2 logistic stack C=1.0 fit on 512 disjoint train calibration examples; decision threshold selected from five-fold out-of-fold calibration probabilities only. Official dev used for reporting; family was explored after other dev results, so cross-family comparison is exploratory. Hidden score unknown.. Run the original builder code/build_b1_stack6_cvcal_submission.py with the exact model probability arrays named and hashed in evidence/VERIFIED_B1_STACK6_CVCAL.json. The record also stores learned coefficients/intercept.

Training objective and validation boundary

There are no optimizer epochs for a new backbone here. The logistic combiner fits probability columns with L2 regularization. The stack with CV-calibrated threshold chooses its cutoff using out-of-fold calibration scores; the other stacks use 0.5. The stack was fitted only on 512 train-calibration rows, and five-fold calibration CV macro-F1 was 0.716744. Official dev macro-F1 0.701977 on 994 cases is exploratory because multiple stack families were compared after seeing dev. The hidden Codabench score is a separate user-supplied observation; calibration CV is not an unseen test result.

Measured results

Five-fold train-calibration CV macro-F1 0.716744 on 512 cases. Exploratory official 994-case dev macro-F1 0.701977. Verified 1,517-ID ZIP SHA-256 e879cdf4f9bc3bef32baa398e4fd931a765589297f7899a2987916f663538ef4. User-provided Codabench screenshot: 0.6651 (finished submission).

Source and evidence trail

The original builder script and verified input-array hashes are in code/ and evidence/. Original base-model probability arrays and checkpoint backups remain in the local B1 study archive. The code SHA-256 manifest, original run records, selected-checkpoint identity, and verified prediction archive are indexed in evidence/README.md. This report is a later reconstruction of preserved evidence, not a claim that its companion notebook ran the original training.

Prediction contract and verification

The original prediction ZIP contains one root-level prediction.json with 1,517 unique official test IDs and integer labels restricted to 0 and 1. This package checks the ZIP CRC and SHA-256 against the archived run/verification record. Full model checkpoint tensors remain outside GitHub; their downloaded backup receipts and hashes live in the local Phase 4 study archives.

Reproducibility and limits

The stack was explored after inspecting other dev results, so comparisons across stack families on the same official dev set are post-selection and may be optimistic. The train-calibration CV is not an unseen test estimate. Reproduction needs the saved per-model probability arrays and selected checkpoint lineage in the original local B1 archive; the small package includes the builder script and full hash/provenance record, but not all probabilities or model weights. Reports and companion notebook were created after training; original scores come from copied artifacts. Model weights and course data are preserved locally, not uploaded to GitHub.